VLSI DESIGN INTERNSHIP PROGRAM

May – June 2025

The report is based on the work carried out during the summer internship in

AHB to APB Bridge RTL Design

By :Akshay konkepudi

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

TABLE OF CONTENTS

ABSTRACT

This work involves designing and implementation of AHB to APB Bridge using Verilog HDL. An AHB to APB Bridge is an important building block in ARM AMBA based System-on-Chip (SoC) design architectures where a high speed Advanced High-performance Bus (AHB) is used along with a low power Advanced Peripheral Bus (APB) and the bridge facilitates the communication between both the buses translating AHB transactions into respective APB transactions and vice versa.

The Architecture comprises mainly of two parts, AHB Slave Interface and APB Controller. The AHB Slave Interface takes in the AHB information such as address, write data and the control information, validates the transaction and passes on the required information to the APB controller. The APB Controller generates the required APB control signals like PSEL, PENABLE, PADDR and PWRITE and controls the transfer process through a finite state machine (FSM) as per APB protocol requirements.

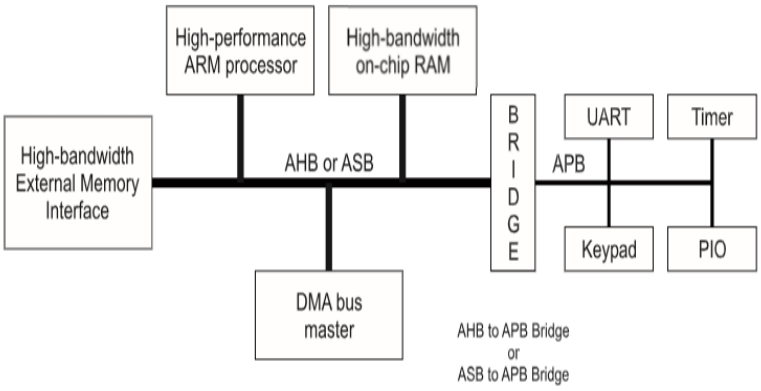
The whole bridge was designed in Verilog HDL and simulation of the same was carried out using Modelsim tool and it was found that the AHB transactions are correctly translated into APB Transactions.

AHB-to-APB Bridge-Design

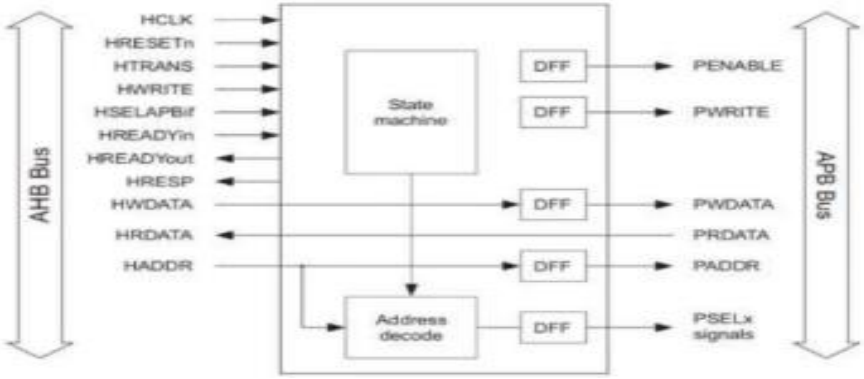
What are AMBA buses?

The AMBA (Advanced Microcontroller Bus Architecture) buses are an array of communication buses that have been designed by ARM and allow for efficient data transfer between processors, memory, and peripherals in SoCs.

- Advanced High-performance Bus (AHB)
- Advanced System Bus (ASB)
- Advanced Peripheral Bus (APB).



AHB to APB Bridge block diagram:



TERMINOLOGY:

Advanced High-performance Bus (AHB)

The AMBA AHB bus serves high-performance and high-frequency system modules. AHB bus serves as the high-performance backbone system bus. AHB is capable of providing a mechanism for connecting processors, on-chip memory, and external memory interfaces with low-power peripheral macrocell functions. AHB is specified such that its use in the design flow will be efficient by means of synthesis and automated test techniques.

Advanced System Bus (ASB)

The AMBA ASB bus is for high-performance system modules. ASB is an alternative system bus which could be used when the high-performance aspects of the AHB bus are not necessary. ASB provides a mechanism for connecting processors, on-chip memory, and external memory interfaces with low-power peripheral macrocell functions.

Advanced Peripheral Bus (APB)

The AMBA APB is for low-power peripherals. APB bus is optimized for low power consumption and reduced interface complexity to provide peripheral functions. APB can be used in combination with any one of the two types of system buses.

Bus cycle

It is the complete sequence of operations which is required to transfer data between a master and a slave device over a communication bus.

Bus transfer

An AMBA ASB or AHB bus transfer is a read or write operation of a data object, which may take one or more bus cycles. The bus transfer is terminated by a completion response from the addressed slave. An AMBA APB bus transfer is a read or write operation of a data object, which always requires two bus cycles.

Burst operation

A burst operation is defined as one or more data transactions, initiated by a bus master, which have a consistent width of transaction to an incremental region of address space. The increment step per transaction is determined by the width of transfer (byte, halfword, word). No burst operation is supported on the APB.

AMBA AHB Signals

Table AMBA AHB signals

Name	Source	Description
HWDATA[31:0] *Write data bus*	Master	Transfers data from the master to the bus slaves during write operations. Minimum recommended width is 32 bits, but it can be extended for higher bandwidth.
HSELx *Slave select*	Decoder	Each AHB slave has its own select signal, indicating the current transfer is intended for it. This is a simple combinatorial decode of the address bus.
HRDATA[31:0] *Read data bus*	Slave	Transfers data from a bus slave to the master during read operations. Minimum

Name	Source	Description
		recommended width is 32 bits, extendable for higher bandwidth.
HREADY *Transfer done*	Slave	When high, indicates a transfer has finished on the bus; driven low to extend a transfer. Every slave must treat HREADY as both an input and an output signal.
HRESP[1:0] *Transfer response*	Slave	Reports the status of a transfer. Four possible responses: OKAY, ERROR, RETRY, SPLIT.

Table for APB AMBA signals:

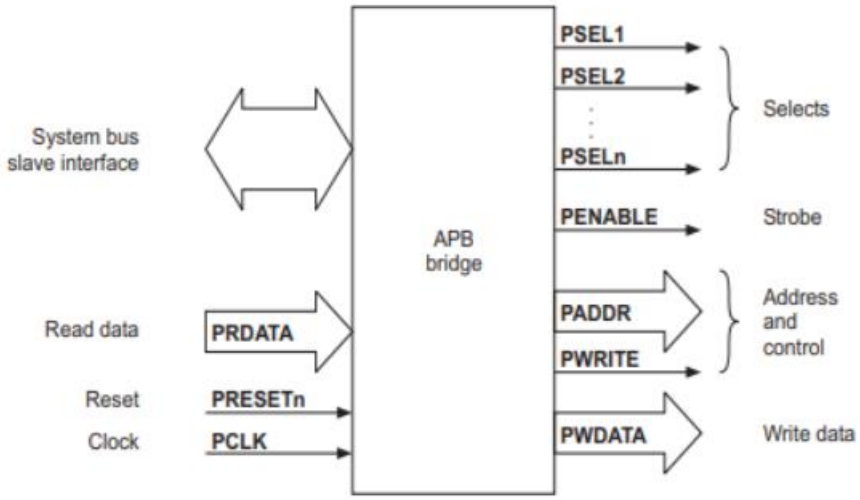
Name	Source	Description
PCLK ***Clock***	**Clock source**	**The rising edge of PCLK times all transfers on the APB bus. This is the same clock used by the AHB or AXI bus that the bridge connects to.**
PRESETn ***Reset***	**System bus**	**The APB peripheral reset signal, active LOW. This is normally connected directly to the system bus reset signal.**
PADDR[31:0] ***Address bus***	**Bridge**	**The APB address bus, driven by the bridge. Width can be up to 32 bits, though most peripherals decode only a subset.**
PSEL ***Select***	**Bridge**	**Driven by the bridge to each peripheral individually (one PSEL per slave). Indicates that the peripheral is selected and that a data transfer is required.**

Name	Source	Description
PENABLE *Enable*	**Bridge**	**Indicates the second and subsequent cycles of an APB transfer. Asserted in the ENABLE state after being LOW during SETUP.**
PWRITE *Direction*	**Bridge**	**Indicates the direction of the transfer. HIGH for a write access, LOW for a read access.**
PWDATA[31:0] *Write data bus*	**Bridge**	**Driven by the bridge during write cycles, valid for the whole transfer whenever PWRITE is HIGH.**
PREADY *Ready*	**Slave**	**Used by the selected peripheral to extend an APB transfer by driving PREADY LOW. When HIGH it indicates the transfer will complete on the next rising clock edge.**
PRDATA[31:0] *Read data bus*	**Slave**	**Driven by the selected peripheral during read cycles, valid while PSEL and PENABLE are both asserted and PWRITE is LOW.**
PSLVERR *Transfer error*	**Slave**	**Indicates a failed transfer. Not all peripherals support this signal; support is optional per the APB protocol.**

What is the main objective of this report ?

The objective of this project is to design and implement an AHB-to-APB Bridge by integrating the AHB Slave Interface and APB Controller to enable efficient communication between AHB and APB buses.

This block diagram below explain the implementation:



The submodules included in this bridge block are :

AHB Slave Interface and APB Controller

Code AHB Slave Interface

```
module ahb_slave_interface(
    input HCLK,
    input HRESETn,
    input HSEL,
    input HWRITE,
    input [31:0] HADDR,
    input [31:0] HWDATA,
    input [1:0] HTRANS,

    output reg [31:0] addr_reg,
```

```
    output reg [31:0] data_reg,
    output reg write_reg,
    output reg valid_transfer
);

always @(posedge HCLK or negedge HRESETn)
begin
    if(!HRESETn)
    begin
        addr_reg <= 32'd0;
        data_reg <= 32'd0;
        write_reg <= 1'b0;
        valid_transfer <= 1'b0;
    end
    else
    begin
        if(HSEL && HTRANS[1])
        begin
            addr_reg <= HADDR;
            data_reg <= HWDATA;
            write_reg <= HWRITE;
            valid_transfer <= 1'b1;
        end
        else
```

```
        valid_transfer <= 1'b0;
    end
end

endmodule
```

Code APB controller

```
module AkshayAc (
    input wire hclk,
    input wire hresetn,


    input wire [31:0] paddr_temp,
    input wire [31:0] pwdata_temp,
    input wire pwrite_temp,
    input wire psel_temp,
    input wire penable_temp,
    output reg hreadyout_temp,


    output reg [31:0] paddr_out,
    output reg [31:0] pwdata_out,
```

```
    output reg pwrite_out,
    output reg psel_out,
    output reg penable_out,
    input wire pready
);


    localparam S_IDLE   = 2'b00;
    localparam S_SETUP  = 2'b01;
    localparam S_ACCESS = 2'b10;

    reg [1:0] state, next_state;


    always @(posedge hclk or negedge hresetn)
    begin
        if(!hresetn)
            state <= S_IDLE;
        else
            state <= next_state;
    end


    always @(*)
```

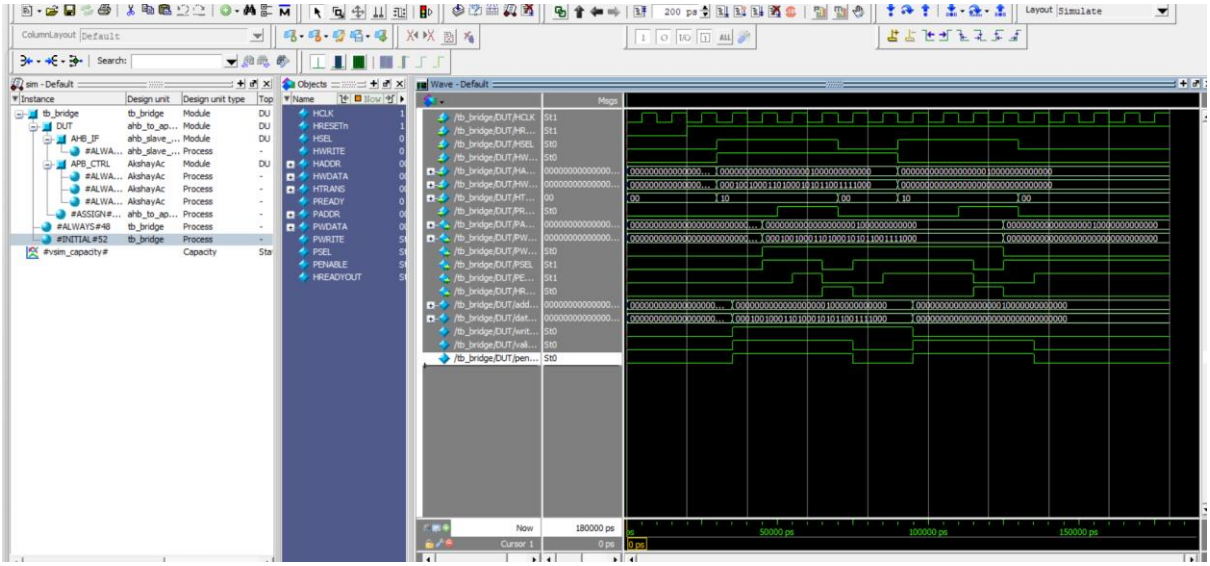
```
begin
   next_state = state;

   case(state)

      S_IDLE:
         if(psel_temp && penable_temp)
            next_state = S_SETUP;

      S_SETUP:
         next_state = S_ACCESS;

      S_ACCESS:
         if(pready)
            next_state = S_IDLE;

      default:
         next_state = S_IDLE;

   endcase
end


always @(posedge hclk or negedge hresetn)
```

```
begin
   if(!hresetn)
   begin
      paddr_out      <= 32'h00000000;
      pwdata_out     <= 32'h00000000;
      pwrite_out     <= 1'b0;
      psel_out       <= 1'b0;
      penable_out    <= 1'b0;
      hreadyout_temp <= 1'b0;
   end
   else
   begin

      case(state)

         S_IDLE:
         begin
            hreadyout_temp <= 1'b0;
            penable_out    <= 1'b0;

            if(psel_temp && penable_temp)
            begin
               paddr_out  <= paddr_temp;
               pwdata_out <= pwdata_temp;
```

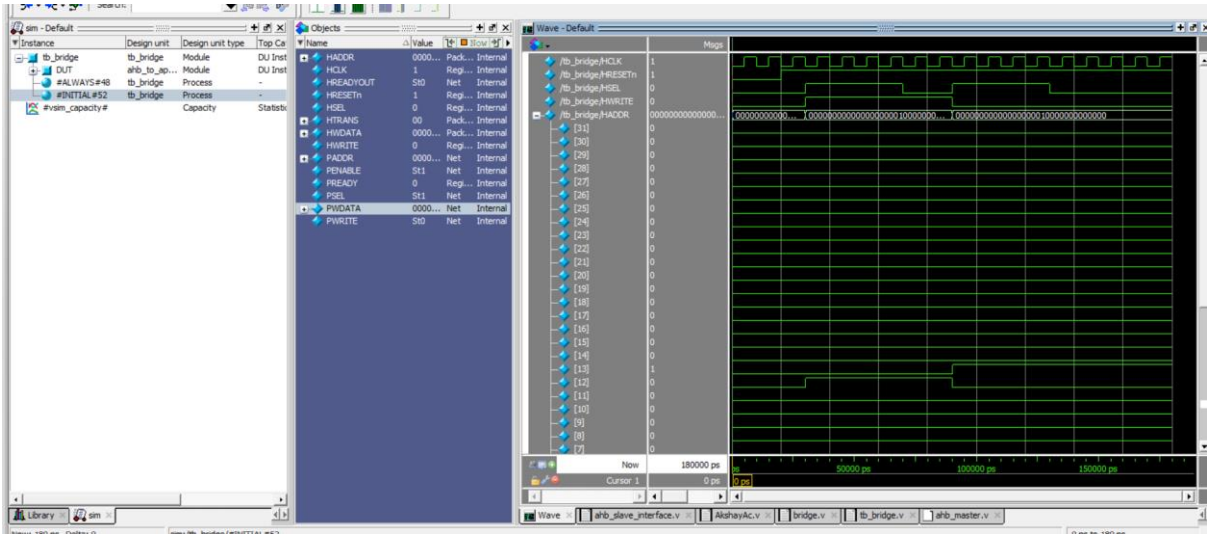
```
            pwrite_out <= pwrite_temp;
            psel_out   <= 1'b1;
        end
    end

    S_SETUP:
    begin
        penable_out <= 1'b1;
    end

    S_ACCESS:
    begin
        if(pready)
        begin
            hreadyout_temp <= 1'b1;
            psel_out       <= 1'b0;
            penable_out    <= 1'b0;
        end
    end

   default:
    begin
    end
endcase
```

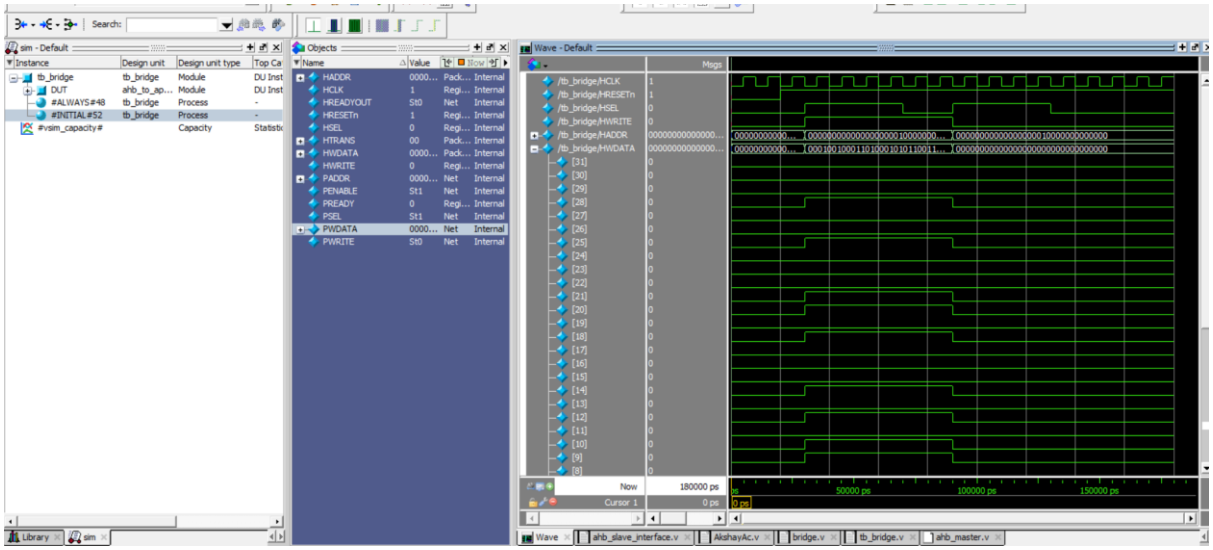
```
        end
    end
endmodule
```

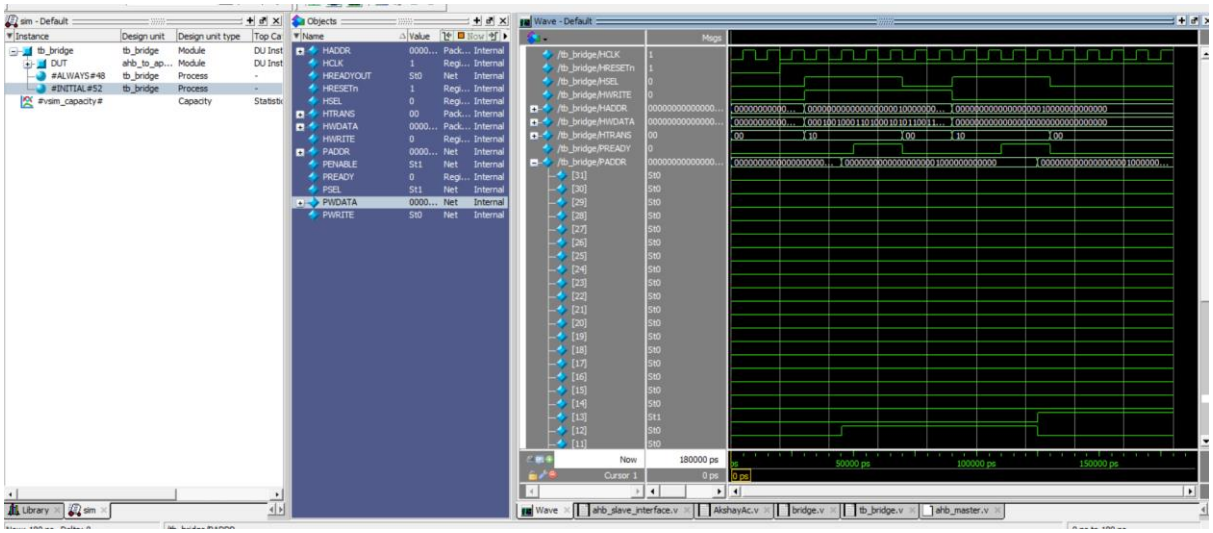
Waveform of AHB slave and APB controller

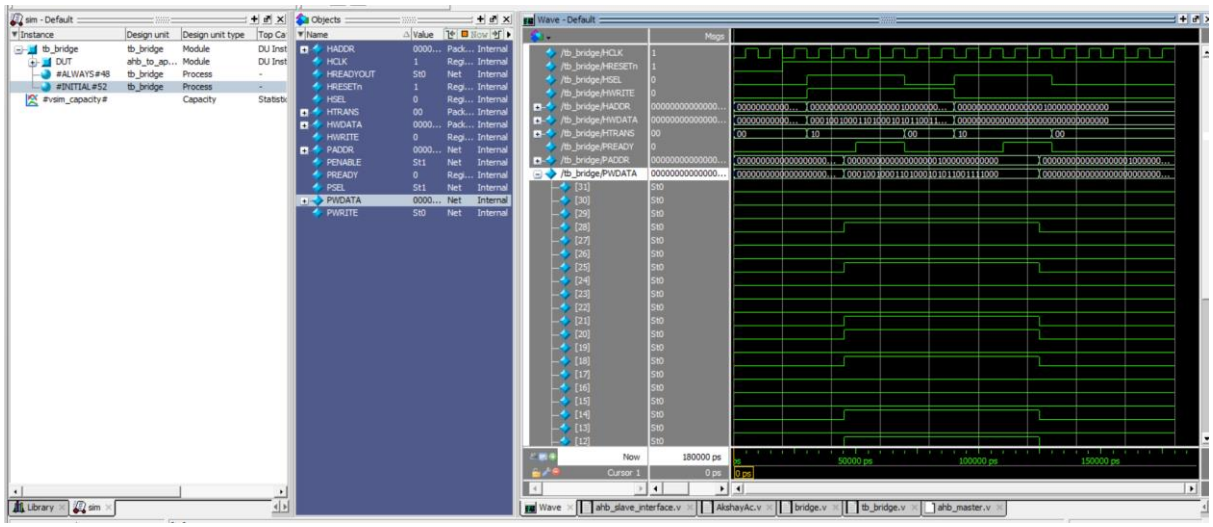


(Simulation waveform showing the operation of the AHB Slave Interface and APB Controller modules integrated within the Bridge Top module)

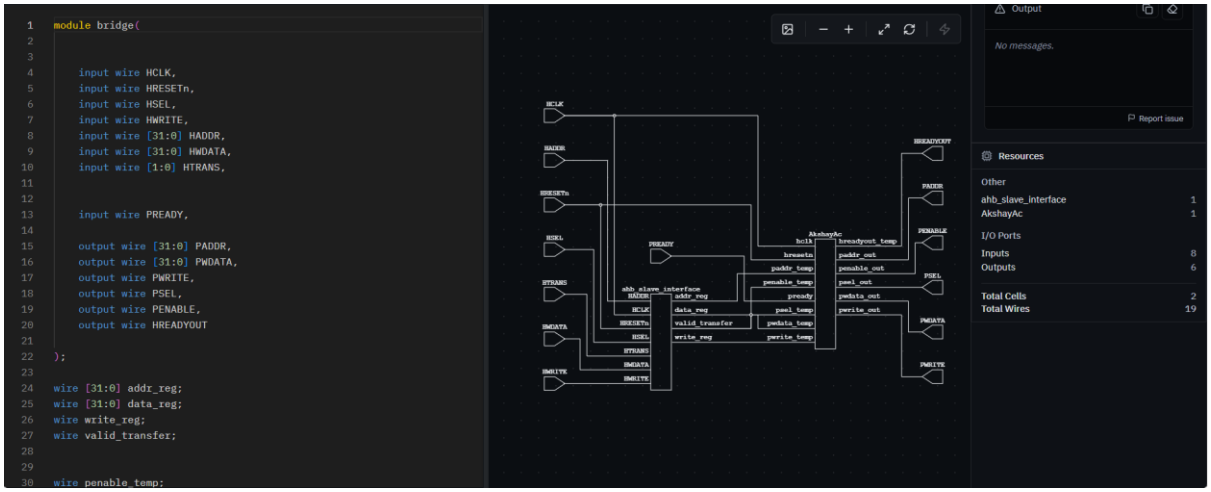


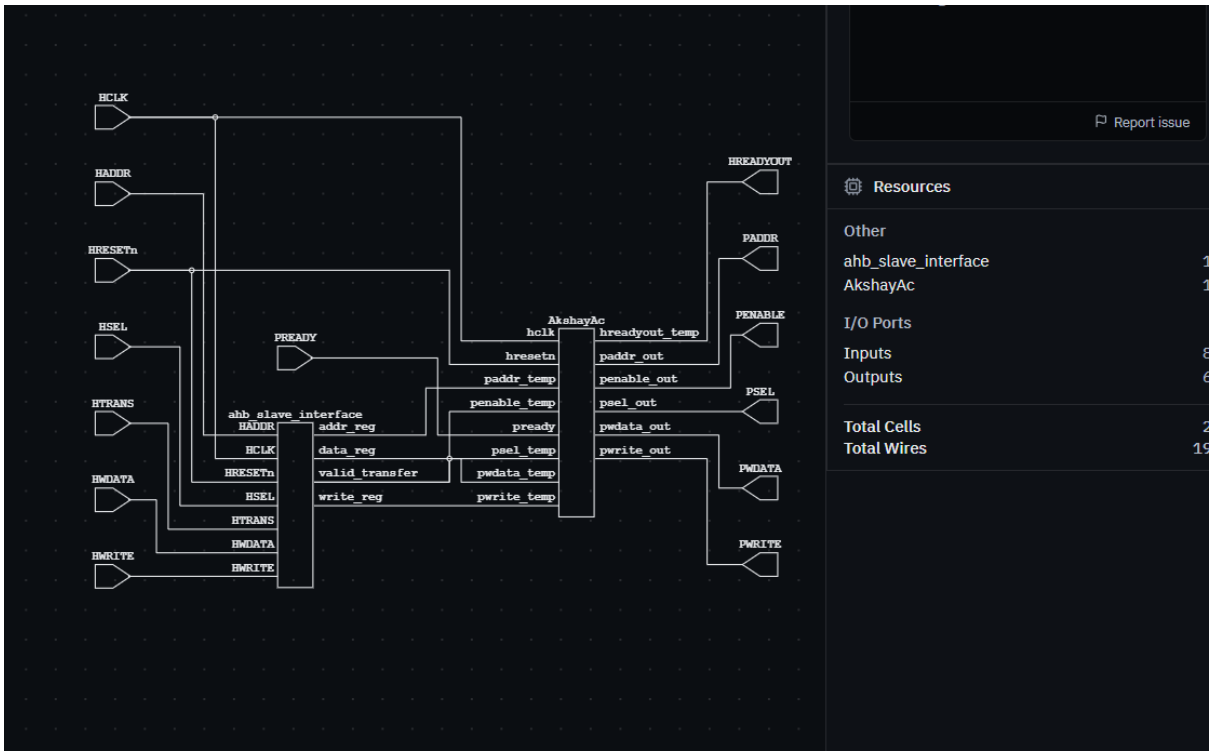






Synthesis





Conclusion

The AHB-to-APB Bridge RTL Design was successfully implemented using Verilog HDL by integrating the AHB Slave Interface and APB Controller modules. Functional verification through simulation confirmed the correct transfer of data and control signals between the high-speed AHB bus and the low-speed APB bus. The project provided practical experience in RTL design, bus protocol

implementation, and digital hardware verification, demonstrating the fundamental concepts involved in AMBA-based on-chip communication.